Polymarket Copy Trading Masterclass

The Complete Guide to Automating Your Prediction Market Profits

---

Price: $65 | Time to Setup: 30-90 minutes | Capital Required: $100+

This guide will teach you how to build a bot that automatically copies the best Polymarket traders. You can follow the steps yourself, or hand this entire guide to an AI agent (Claude Code, OpenClaw, Cursor) to automate the setup.

Table of Contents

- Quick Start (30 Minutes)
- Chapter 1: Polymarket 101
- Chapter 2: The Copy Trading Strategy
- Chapter 3: Understanding the Leaderboard
- Chapter 4: Setting Up Your Bot
- Chapter 5: Configuration & Risk Management
- Chapter 6: Advanced — Signal Integration
- Chapter 7: Monitoring & Operations
- Chapter 8: Giving This Guide to Your AI Agent
- Appendix A: Glossary
- Appendix B: Troubleshooting
- Appendix C: Code Reference

Quick Start (30 Minutes)

Want to skip the theory and get running immediately? Here's the fast track:

Prerequisites

- Python 3.12+ installed
- uv package manager (`curl -LsSf https://astral.sh/uv/install.sh | sh`)
- A Polymarket account (create at polymarket.com)
- $100+ in USDC on Polygon network

Fast Setup

1. **Create a fresh wallet for your bot** (NEVER use your main wallet's private key):

```
# Generate a new private key - save this securely!
openssl rand -hex 32
```

2. **Clone and configure the bot**:

```
git clone https://github.com/your-repo/polymarket-copytrader.git
cd polymarket-copytrader

cp .env.example .env
# Edit .env with your values:
# - PK=<your fresh private key from step 1>
```

```
# - BROWSER_ADDRESS=<your Polymarket wallet address>
# - COPY_TRADE_TARGET=<username or 0x address from leaderboard>
```

3. **Install and run**:

```
uv sync
uv run python scripts/init_sqlite_db.py
uv run python scripts/run_copytrade.py
```

4. **Fund your bot wallet**: Send at least $100 USDC on Polygon to your new bot wallet address
5. **Verify it's working**: Check Polymarket UI for positions appearing

Done! Your bot is now copying trades. Read the rest of this guide to understand what you just built and how to optimize it.

Chapter 1: Polymarket 101

What Are Prediction Markets?

Prediction markets are platforms where people bet on the outcomes of real-world events. Will Bitcoin hit $100k by 2025? Will a certain candidate win an election? The market aggregates collective knowledge into a price that reflects the probability of an event occurring.

Why Prediction Markets Are Lucrative:

Unlike traditional markets (stocks, crypto), prediction markets are:

- **Inefficient**: Fewer participants = more mispricing
- **Binary**: Outcomes are usually yes/no, simplifying analysis
- **Niche**: Most traders don't understand them well
- **Information-dense**: Prices update constantly with new information

This creates opportunity: Smart traders who consistently find mispriced markets can generate significant profits. Your bot copies these traders automatically.

How Polymarket Works

Polymarket is the largest prediction market platform, built on Polygon (a scaling solution for Ethereum). Here's the architecture:

The Basics:

- **Binary markets**: Most markets have two outcomes: YES and NO
- **CLOB (Central Limit Order Book)**: Like stock exchanges, you can place limit orders
- **USDC collateral**: All bets are settled in USDC (a stablecoin pegged to $1)
- **Polygon network**: Fast, cheap transactions (~$0.01 per trade)

An Example Market:

> *"Will Bitcoin exceed $100,000 by December 31, 2025?"*

- Current YES price: 35¢ (35% probability)
- Current NO price: 65¢ (65% probability)
- If you buy YES at 35¢ and Bitcoin hits $100k, you get $1 per share (186% profit)

- If not, your YES shares become worthless (100% loss)

The Key Insight: If you consistently buy undervalued YES shares (or NO shares), you'll profit over time. Top traders on Polymarket are experts at finding these mispricings.

Creating and Funding Your Account

Step 1: Create a Polymarket Account

1. Go to polymarket.com
2. Click "Sign Up" and connect a wallet (MetaMask, WalletConnect, etc.)
3. **Important**: Polymarket uses a "proxy wallet" system — your connected wallet creates a Polymarket-specific address for trading

Step 2: Get USDC on Polygon

You need USDC on the Polygon network (not Ethereum mainnet — fees are too high).

Option A: Buy Directly (Easiest)

1. Use Coinbase, Binance, or Kraken
2. Buy USDC
3. Withdraw to your wallet using the **Polygon** network

Option B: Bridge from Ethereum

1. Buy USDC on Ethereum mainnet
2. Use a bridge (like Across or Stargate)
3. Bridge USDC from Ethereum → Polygon

Expected Cost: ~$0.50 in bridging fees, regardless of amount

Step 3: Deposit on Polymarket

1. Click "Deposit" in the top-right
2. Enter amount (start with at least $100)
3. Confirm the transaction in your wallet

Expected Time: 2-5 minutes

You're Now Ready to Trade! But instead of trading manually, let's build a bot to copy the best traders.

Chapter 2: The Copy Trading Strategy

Why Copy Trading Works

In any market, a small percentage of participants consistently outperform. They have:

- Better information
- Superior analysis
- More experience
- Or some combination of the above

Copy trading leverages their edge: Instead of trying to beat the market yourself, you mirror the positions of proven winners.

Why This Works on Polymarket:

1. **Transparency**: All trades are on-chain — we can see who's winning
2. **Leaderboard**: Polymarket ranks traders by performance

3. **Simple mechanics**: Binary outcomes make copying straightforward
4. **Asymmetric information**: Top traders often research deeper than the market

The Mathematical Edge:

Expected Value (EV) per trade:

```
EV = (Win Rate × Average Win) − (Loss Rate × Average Loss)
```

Example for a top trader:

- Win rate: 65%
- Average win: $50
- Average loss: $30
- EV = (0.65 × $50) - (0.35 × $30) = $32.50 - $10.50 = **$22 per trade**

If they make 10 trades per day, that's $220/day in expected profit. Your bot copies every trade.

Identifying Top Traders

Not all profitable traders are good copy targets. Here's what to look for:

Key Metrics:

1. **Win Rate (% of trades profitable)**
   - Target: 55%+ (anything above 50% is profitable with proper sizing)
   - Red flag: Win rate > 90% (likely cheating or low sample size)
2. **PnL (Profit and Loss)**
   - Total profit in USD
   - Focus on traders with $1,000+ in PnL (proven track record)
   - Negative PnL = avoid, regardless of win rate
3. **Volume (Total amount traded)**
   - Higher volume = more activity to copy
   - Target: $5,000+ in volume
   - Low volume (<$500) = trader might be inactive
4. **Consistency**
   - Check the leaderboard daily snapshots
   - Good targets: Steady PnL growth over weeks
   - Bad targets: One big win, then flat or declining

Red Flags to Avoid:

- **High win rate, negative PnL**: They win small, lose big
- **Low volume, high PnL**: Lucky one-off bet
- **Recent join date, massive PnL**: Could be a washed-up account
- **Inconsistent activity**: Trades once per week = hard to copy

Good Copy Target Example:

Rank	Trader	Win Rate	W/T	PnL	Volume

#12	prediction_wizard	62%	145/93	+$4,230	$12,400

This trader has:

- ✅ Solid win rate (62%)
- ✅ Positive PnL (+$4,230)
- ✅ Healthy volume ($12,400)
- ✅ Consistent activity (238 trades)

Bad Copy Target Example:

Rank	Trader	Win Rate	W/T	PnL	Volume
#47	lucky_shot	89%	8/1	+$120	$150

Avoid because:

- ❌ Tiny sample size (9 trades)
- ❌ Low volume ($150)
- ❌ Could be a fluke

Risk vs Reward

Not all high win-rate traders are profitable. Here's why:

Example A: The "Small Win, Big Loss" Trader

- Win rate: 80%
- Average win: $10
- Average loss: $100
- EV = (0.80 × $10) - (0.20 × $100) = $8 - $20 = **-$12 per trade**

High win rate, but **losing money**.

Example B: The "Calculated Risk" Trader

- Win rate: 55%
- Average win: $100
- Average loss: $50
- EV = (0.55 × $100) - (0.45 × $50) = $55 - $22.50 = **+$32.50 per trade**

Lower win rate, but **profitable**.

The Lesson: Always look at PnL, not just win rate. The leaderboard shows both.

Chapter 3: Understanding the Leaderboard

How the Leaderboard Scores Traders

Polymarket's leaderboard ranks traders by a combination of factors:

Primary Metrics:

1. **PnL Rank (#)**: Your position on the leaderboard
2. **Username**: Display name (can be searched)
3. **Win Rate (%)**: Percentage of profitable trades
4. **W/T**: Wins divided by total trades

5. **PnL**: Total profit/loss in USD
6. **Volume**: Total amount traded in USD

Accessing the Leaderboard:

1. Go to polymarket.com
2. Click "Leaderboard" in the navigation
3. Filter by time period (24h, 7d, 30d, All Time)

Pro Tip: Use the "All Time" filter to find consistent performers. 24h and 7d rankings can be noisy.

Reading the Data

Leaderboard Entry Format:

```
Rank: #12
PnL#: #8
Username: prediction_wizard
Win%: 62%
W/T: 145/93
PnL: +$4,230
Volume: $12,400
```

What Each Field Means:

- **Rank (#12)**: Current position on leaderboard
- **PnL# (#8)**: Rank by pure profit (different from overall rank)
- **Username**: Searchable identifier
- **Win% (62%)**: 62% of trades are profitable
- **W/T (145/93)**: 145 winning trades out of 238 total
- **PnL (+$4,230)**: Net profit in USD
- **Volume ($12,400)**: Total amount traded

Filtering for Crypto-Specific Markets

Many traders specialize. You might want to copy only:

- Crypto markets
- Politics markets
- Sports markets

How to Filter:

1. Use the leaderboard API (see Appendix C for code)
2. Filter by market tags or keywords
3. Check a trader's history manually by clicking their profile

Example: Finding Crypto Specialists

```
# Pseudo-code for filtering crypto traders
crypto_traders = [
    t for t in leaderboard
    if t.avg_market_category == "crypto"
    and t.pnl > 1000
]
```

Daily Snapshots: Tracking Consistency

The leaderboard is a snapshot in time. Smart copy traders track targets over time.

What to Track:

1. **Daily PnL changes**: Is the target consistently profitable?
2. **Volume trends**: Are they still active?
3. **Win rate stability**: Has their performance changed?

How to Track:

- Manually: Check the leaderboard daily and record in a spreadsheet
- Automatically: Use the bot's built-in database (see Chapter 4)

Red Flags

Warning Signs to Watch For:

1. **Sudden PnL Drop**
   - Target might be experimenting with new strategy
   - Consider pausing the bot
2. **Volume Inactivity**
   - If volume hasn't changed in 24h, target stopped trading
   - Bot will have nothing to copy
3. **Win Rate Fluctuation**
   - Win rate dropping below 50%? Consider finding a new target
4. **Account Changes**
   - Target changes username or disappears from leaderboard
   - Update your `.env` with a new target

Chapter 4: Setting Up Your Bot

Prerequisites

Before starting, ensure you have:

Required:

- **Python 3.12+**: Download from python.org
- **uv package manager**: Install with:

  ```
  curl -LsSf https://astral.sh/uv/install.sh | sh
  ```

- **A Polymarket account**: From Chapter 1
- **USDC on Polygon**: At least $100 to start

Recommended:

- **Screen or tmux**: For running the bot in background
- **Systemd knowledge**: For auto-start on boot (Linux)

- **Basic terminal familiarity**: You'll be running commands

Creating a New Wallet for Your Bot

⚠️ **CRITICAL SECURITY WARNING**

NEVER use your main wallet's private key for the bot. If the bot is compromised or bugs out, you don't want your main funds at risk.

Step 1: Generate a Fresh Private Key

```
# Generate a secure random private key
openssl rand -hex 32
```

Expected output:

```
a1b2c3d4e5f6...64 character hex string
```

Step 2: Save This Securely

- Store in a password manager (1Password, Bitwarden)
- Write it down on paper and store in a safe
- **Never share this key or commit it to git**

Step 3: Get Your Wallet Address

You'll need the public address (derived from the private key) for funding.

Option A: Use Python

```python
from eth_account import Account
import os

# Generate account from private key
private_key = "0x" + os.environ['PK']
account = Account.from_key(private_key)
print(account.address)  # Your wallet address
```

Option B: Use an online tool (risky but convenient)

- Go to app.mycrypto.com
- Click "View & Send" → "Add Account"
- Enter your private key
- Copy the displayed address
- **Clear your browser history after**

Step 4: Fund Your Bot Wallet

Send at least $100 USDC (Polygon) to your bot's public address.

Expected time: 2-5 minutes for confirmation

Step-by-Step: Clone and Configure

Step 1: Clone the Repository

```
git clone https://github.com/your-repo/polymarket-copytrader.git
cd polymarket-copytrader
```

Expected output:

```
Cloning into 'polymarket-copytrader'...
remote: Enumerating objects: 150, done.
remote: Total 150 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (150/150), 45.00 KiB | 1.2 MiB/s, done.
```

Step 2: Install Dependencies

```
uv sync
```

Expected output:

```
Resolved 23 packages in 2.4s
Downloaded 23 packages in 1.2s
Installed 23 packages in 0.8s
```

Step 3: Configure Environment Variables

```
cp .env.example .env
nano .env  # Or use your preferred editor
```

Environment Variables Explained:

Variable	Purpose	Example Value	Required
`PK`	Bot wallet private key	`0x1234...`	✅ Yes
`BROWSER_ADDRESS`	Your Polymarket wallet	`0xabcd...`	✅ Yes
`COPY_TRADE_TARGET`	Trader to copy	`prediction_wizard` or `0x1234...`	✅ Yes
`COPY_TRADE_SIZE_MULTIPLIER`	Scale position size	`0.5` (half size) or `1.0` (same size)	✅ Yes
`MIN_SIZE`	Minimum trade size (USD)	`5`	❌ No (default: 1)
`MAX_POSITION_USD`	Max position per market	`50`	❌ No (default: 100)
`COPY_TRADE_POLL_SECONDS`	Check frequency	`60`	❌ No (default: 30)
`COPY_TRADE_MARKET_FILTER`	Filter markets by tag	`crypto`	❌ No (default: all)
`COPY_TRADE_IGNORE_MARKETS`	Markets to skip	`market_id_1,market_id_2`	❌ No (default: none)

Example .env File:

```
# Bot Wallet (NEVER share this!)
PK=0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef

# Your main Polymarket wallet (for withdrawal/monitoring)
```

```
BROWSER_ADDRESS=0xabcdabcdabcdabcdabcdabcdabcdabcdabcdabcd

# Copy Trading Configuration
COPY_TRADE_TARGET=prediction_wizard
COPY_TRADE_SIZE_MULTIPLIER=0.5
MIN_SIZE=5
MAX_POSITION_USD=50

# Polling (how often to check for new trades)
COPY_TRADE_POLL_SECONDS=60

# Optional: Focus only on crypto markets
COPY_TRADE_MARKET_FILTER=crypto
```

Finding Your Values:

1. **PK**: The private key you generated earlier
2. **BROWSER_ADDRESS**: Your main Polymarket wallet address (found in Polymarket settings)
3. **COPY_TRADE_TARGET**: A username from the leaderboard OR a 0x address

Step 4: Initialize the Database

```
uv run python scripts/init_sqlite_db.py
```

Expected output:

```
✅ Database initialized: bot_data/trades.db
✅ Created tables: positions, trades, leaderboard_snapshots
```

Step 5: Run the Bot

```
uv run python scripts/run_copytrade.py
```

Expected output:

```
🚀 Starting Polymarket Copy Trade Bot
────────────────────────────────────────────────────────────
📊 Config:
  Target: prediction_wizard (0x1234567890123456789012345678901234567890)
  Size multiplier: 0.5x
  Max position: $50
  Polling: every 60s

🔗 Connected to Polymarket CLOB
💰 Bot wallet: 0x9876543210987654321098765432109876543210
💰 USDC Balance: $100.00

👀 Watching for trades from prediction_wizard...
[2025-01-15 10:23:45] 🔍 Checking for new positions...
[2025-01-15 10:23:47] ✅ Found position: YES @ 0.35 in market 0xabc...
[2025-01-15 10:23:48] 📈 Placing mirrored order: $17.50 worth of YES @ 0.35
[2025-01-15 10:23:50] ✅ Order filled! Position opened.
```

What the Bot Does Under the Hood

Here's the technical flow of how the copy trading bot works:

1. Authentication

```
Your PK → Derive L2 credentials (Polymarket CLOB auth)
```

The bot derives specialized credentials for Polymarket's CLOB (Central Limit Order Book) from your private key. This is similar to how Polymarket's web app works.

2. Polling the Leaderboard

```
Every 60 seconds → Fetch target's positions from Polymarket API
```

The bot checks if your target has opened any new positions.

3. Position Mirroring

```
Target buys YES @ 0.35 → Bot buys YES @ 0.35 (scaled by multiplier)
Target buys $100 worth → Bot buys $50 worth (with 0.5x multiplier)
```

The bot calculates the position size based on your `COPY_TRADE_SIZE_MULTIPLIER`.

4. Order Placement

```
Create limit order → Sign with derived credentials → Submit to CLOB
```

Orders are signed using your derived credentials and submitted to Polymarket's order book.

5. Tracking

```
All positions saved to SQLite database (bot_data/trades.db)
```

The bot tracks all trades for PnL calculation and debugging.

Key Files:

- `polymarket_client.py` : Handles Polymarket API interaction
- `copytrade.py` : Core copy trading logic
- `run_copytrade.py` : Main bot loop

(See Appendix C for detailed code reference)

Chapter 5: Configuration & Risk Management

Sizing: Start Small

Golden Rule: Start with small position sizes and increase gradually as you verify the bot works.

Recommended Starting Configuration:

```
COPY_TRADE_SIZE_MULTIPLIER=0.1   # Copy 10% of target's position size
MIN_SIZE=5                        # Minimum $5 per trade
```

```
MAX_POSITION_USD=20              # Maximum $20 per market
```

Example:

If your target buys $500 worth of YES @ 0.40:

- Your bot buys: $500 × 0.1 = **$50 worth**
- But `MAX_POSITION_USD=20`, so bot buys **$20 worth** instead

Why Start Small?

1. **Testing**: Verify the bot works correctly
2. **Learning**: Understand how the target trades
3. **Risk management**: Limit losses if the target has a bad week
4. **Confidence**: Build confidence in the system

When to Increase:

- After 1 week of consistent operation
- If the target's win rate remains stable
- If you're profitable (even slightly)

Suggested Progression:

Week	Multiplier	Max Position	Max Total Exposure*
1	0.1x	$20	$100
2	0.25x	$30	$150
3	0.5x	$50	$250
4+	1.0x	$100	$500

*Assumes 5 concurrent positions max

Position Limits

`MAX_POSITION_USD`: Maximum amount to invest in a single market

Why This Matters:

Prediction markets can go to 0. If you put $500 into a NO position and the event happens, you lose $500.

Recommended Settings:

Total Capital	Max Position	Rationale
$100	$10	10% per position
$500	$50	10% per position
$1,000	$100	10% per position
$5,000+	$250	5% per position

Formula:

```
max_position = total_capital * 0.10  # 10% rule
```

Polling Frequency

`COPY_TRADE_POLL_SECONDS` : How often to check for new trades

Trade-offs:

Frequency	Pros	Cons
10s	Catch trades immediately	More API calls, higher rate limit risk
30s (default)	Good balance	Minimal
60s	Lower rate limit risk	Might miss fast-moving markets
300s	Very low API usage	Too slow, will miss trades

Recommendation: Start with `30` seconds. If you hit rate limits, increase to `60` .

Rate Limit Symptoms:

- Bot logs show "429 Too Many Requests"
- Positions not being copied

Market Filtering

`COPY_TRADE_MARKET_FILTER` : Filter by market category

Examples:

```
# Only crypto markets
COPY_TRADE_MARKET_FILTER=crypto

# Only politics
COPY_TRADE_MARKET_FILTER=politics

# No filter (copy all markets)
# COPY_TRADE_MARKET_FILTER=  (leave empty)
```

`COPY_TRADE_IGNORE_MARKETS` : Skip specific markets

```
# Comma-separated list of market IDs or tokens
COPY_TRADE_IGNORE_MARKETS=will-bitcoin-hit-100k,us-election-2024
```

Why Filter?

1. **Expertise**: Your target might be amazing at crypto but terrible at politics
2. **Risk tolerance**: Some markets are more volatile
3. **Capital efficiency**: Focus on markets you understand

Multiple Targets

You can copy multiple traders by comma-separating them:

```
COPY_TRADE_TARGET=prediction_wizard,market_master,crypto_king
COPY_TRADE_SIZE_MULTIPLIER=0.33
```

How It Works:

- The bot copies ALL targets
- Position size is divided by number of targets
- Example: With 3 targets and 0.33x multiplier, each trade is 11% of original (0.33 / 3)

Risk: More targets = more complexity = more potential for errors

Recommendation: Start with ONE target, add more after 1-2 weeks.

Stop Rules

When to Pause the Bot:

1. **Drawdown Limit**: If you lose 20% of your capital
    - Manual: Kill the bot process
    - Automatic: Coming in future versions
2. **Target Change**: If your target's performance drops
    - Win rate drops below 50% for 3+ days
    - PnL declines for 5+ days
3. **Technical Issues**:
    - Bot errors increase
    - Positions not being mirrored
    - Exchange issues
4. **Personal Reasons**:
    - You need to withdraw funds
    - Market volatility is too high
    - You're going on vacation

How to Pause:

```
# Press Ctrl+C to stop the bot
# It will cleanly shutdown and save state

# To restart:
uv run python scripts/run_copytrade.py
```

Chapter 6: Advanced — Signal Integration (Optional)

Adding AI-Powered Signals

The basic copy trading bot mirrors positions exactly. Advanced users can add AI-powered signals to:

1. **Validate trades**: Only copy if signals agree
2. **Size positions**: Bigger when signals are strong

3. **Filter markets**: Avoid markets where signals disagree

Signal Sources:

1. Grok Sentiment (xAI)

```
import openai

client = openai.OpenAI(
    base_url="https://api.x.ai/v1",
    api_key=os.environ["XAI_API_KEY"]
)

response = client.chat.completions.create(
    model="grok-2",
    messages=[{
        "role": "user",
        "content": f"Analyze sentiment for: {market_question}. Answer YES, NO, or UNCERTAIN."
    }]
)

sentiment = response.choices[0].message.content
```

2. Gemini Analysis (Google)

```
import google.generativeai as genai

genai.configure(api_key=os.environ["GEMINI_API_KEY"])
model = genai.GenerativeModel('gemini-2.0-flash')

response = model.generate_content(
    f"Will this happen: {market_question}? Consider recent news. Answer YES or NO."
)
```

3. Macroeconomic Data (yfinance)

```
import yfinance as yf

# Get BTC price for crypto markets
btc = yf.Ticker("BTC-USD")
current_price = btc.history(period="1d")['Close'].iloc[-1]

# Use this data to inform crypto market predictions
```

4. Brave Search API (News)

```
import requests

headers = {"X-Subscription-Token": os.environ["BRAVE_API_KEY"]}
response = requests.get(
    "https://api.search.brave.com/res/v1/web/search",
```

```
    params={"q": market_question},
    headers=headers
)

news = response.json().get('web', {}).get('results', [])
```

Signal Aggregation

Weighted Blend Strategy:

```
def calculate_signal_score(market, signals):
    """
    Combine multiple signals into a single score
    Returns: -1 (strong NO) to +1 (strong YES)
    """
    score = 0

    # Copy trading signal (highest weight)
    if signals['copy_trade'] == 'YES':
        score += 0.5
    elif signals['copy_trade'] == 'NO':
        score -= 0.5

    # AI sentiment (medium weight)
    if signals['grok_sentiment'] == 'YES':
        score += 0.25
    elif signals['grok_sentiment'] == 'NO':
        score -= 0.25

    # Macro data (lower weight)
    if signals['macro_trend'] == 'bullish':
        score += 0.15
    elif signals['macro_trend'] == 'bearish':
        score -= 0.15

    # News sentiment (lower weight)
    if signals['news_sentiment'] == 'positive':
        score += 0.10
    elif signals['news_sentiment'] == 'negative':
        score -= 0.10

    return score  # Range: -1.0 to +1.0
```

Usage:

```
score = calculate_signal_score(market, signals)

if score > 0.5:
    # Strong YES signal
    size_multiplier = 1.5
elif score > 0:
```

```
    # Weak YES signal
    size_multiplier = 1.0
elif score < -0.5:
    # Strong NO signal
    size_multiplier = 1.5
else:
    # Weak NO signal
    size_multiplier = 1.0

# Execute trade with adjusted size
place_order(market, side='YES', size=base_size * size_multiplier)
```

API Keys and Costs

Required Keys:

Service	API Key	Cost	Free Tier
Grok (xAI)	x.ai	$2/day	Yes (limited)
Gemini	ai.google.dev	$0.075/1M tokens	Yes (generous)
Brave Search	brave.com	$0.0025/call	2,000 calls/month free
yfinance	N/A	Free	Unlimited

Estimated Daily Cost: ~$2-5 with moderate usage

Setup:

```
# Add to your .env
XAI_API_KEY=your_xai_key_here
GEMINI_API_KEY=your_gemini_key_here
BRAVE_API_KEY=your_brave_key_here
```

Recommendation: Start WITHOUT signals. Only add them after you've run the basic bot for 2+ weeks and understand your target's trading patterns.

Chapter 7: Monitoring & Operations

Checking Positions

Via Polymarket Web UI (Easiest):

1. Go to polymarket.com
2. Connect your bot wallet (or check your `BROWSER_ADDRESS`)
3. Click "Positions" to see open trades

Via Database:

```
sqlite3 bot_data/trades.db "SELECT * FROM positions ORDER BY opened_at DESC LIMIT 10;"
```

Expected output:

```
1|0xabc123...|0xmarket...|YES|0.45|20.00|2025-01-15 10:23:48
2|0xdef456...|0xmarket...|NO|0.30|15.00|2025-01-15 11:45:12
```

Bot Logs: What to Look For

Normal Log Messages:

```
✅ Found position: YES @ 0.35 in market 0xabc...
✅ Order filled! Position opened.
🔍 Checking for new positions...
ℹ️ No new positions found.
```

Warning Signs:

```
❌ ERROR: Failed to place order: Insufficient funds
❌ ERROR: Rate limited. Retrying in 60s...
❌ ERROR: Auth failed. Check PK variable.
```

What to Monitor Daily:

1. **Bot is running**: Check the process is alive
2. **New positions copied**: Are trades being mirrored?
3. **Balance**: Are you gaining or losing USDC?
4. **Errors**: Any red flags in the logs?

Common Errors and Fixes

Error 1: "Insufficient Funds"

Cause: Bot wallet doesn't have enough USDC

Fix:

```
# Check balance
uv run python scripts/check_balance.py

# Send more USDC to bot wallet
```

Error 2: "401 Unauthorized"

Cause: Invalid `PK` or derived credentials

Fix:

```
# Verify PK in .env (should start with 0x)
cat .env | grep PK

# Regenerate credentials
uv run python scripts/init_auth.py
```

Error 3: "429 Rate Limited"

Cause: Too many API calls

Fix:

```
# Edit .env
COPY_TRADE_POLL_SECONDS=60  # Increase from 30

# Restart bot
```

Error 4: "Market Not Found"

Cause: Invalid market ID or market closed

Fix:

```
# Usually resolves itself — bot will skip invalid markets
# If persistent, add to COPY_TRADE_IGNORE_MARKETS
```

Error 5: "Connection Timeout"

Cause: Network issues or Polymarket API down

Fix:

```
# Check internet connection
ping polymarket.com

# If API is down, wait. Bot will retry automatically.
```

Running as a Background Service

Option 1: Screen (Simplest)

```
# Start screen session
screen -S polymarket-bot

# Run bot
uv run python scripts/run_copytrade.py

# Detach: Press Ctrl+A, then D
# Reattach: screen -r polymarket-bot
```

Option 2: tmux

```
# Start tmux session
tmux new -s polymarket-bot

# Run bot
uv run python scripts/run_copytrade.py

# Detach: Press Ctrl+B, then D
# Reattach: tmux attach -t polymarket-bot
```

Option 3: Systemd (Linux - Auto-start on boot)

Create `/etc/systemd/system/polymarket-bot.service`:

```
[Unit]
Description=Polymarket Copy Trade Bot
After=network.target

[Service]
Type=simple
User=your_username
WorkingDirectory=/path/to/polymarket-copytrader
ExecStart=/usr/local/bin/uv run python scripts/run_copytrade.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl enable polymarket-bot
sudo systemctl start polymarket-bot
sudo systemctl status polymarket-bot
```

Daily Checklist

Every Morning (5 minutes):

1. ✅ **Verify bot is running**: `ps aux | grep run_copytrade`
2. ✅ **Check balance**: Polymarket UI or `check_balance.py`
3. ✅ **Review overnight trades**: Database or UI
4. ✅ **Check logs for errors**: `tail -f bot_data/bot.log`

Every Week (15 minutes):

1. ✅ **Calculate PnL**: Total profit/loss
2. ✅ **Review target performance**: Is win rate stable?
3. ✅ **Backup database**: `cp bot_data/trades.db bot_data/backup/`
4. ✅ **Evaluate sizing**: Should you increase `MAX_POSITION_USD`?

Every Month (30 minutes):

1. ✅ **Deep performance review**: Compare target PnL vs your PnL
2. ✅ **Consider new targets**: Are there better traders to copy?
3. ✅ **Update bot**: `git pull && uv sync`
4. ✅ **Security audit**: Rotate private key if needed

Chapter 8: Giving This Guide to Your AI Agent

You don't have to do the setup yourself! AI agents like Claude Code, OpenClaw, and Cursor can read this guide and execute it.

How AI Agents Execute This Guide

The Process:

1. **Read the guide**: Agent ingests this entire Markdown file
2. **Parse commands**: Extracts bash commands, code blocks, and instructions
3. **Execute sequentially**: Runs each command, checking for success
4. **Handle errors**: If a command fails, agent troubleshoots
5. **Report progress**: Updates you on what's been completed

What the Agent CAN Automate:

- ✅ Clone the repository
- ✅ Install dependencies
- ✅ Configure environment variables
- ✅ Generate wallet credentials
- ✅ Initialize the database
- ✅ Start the bot

What the Agent CANNOT Do:

- ❌ Fund your wallet (you must send USDC)
- ❌ Choose your copy target (you must decide)
- ❌ Monitor the bot long-term (you should check daily)

Prompt Template

For Claude Code / OpenClaw / Cursor:

```
Read the file at guide/polymarket-copy-trading-masterclass.md and execute all the setup steps to
create a Polymarket copy trading bot.

Here are my choices:
- Copy target: prediction_wizard (or provide username)
- Starting capital: $500 USDC
- Risk tolerance: Conservative (start with 0.1x multiplier)

After setup, provide me with:
1. The bot wallet address (so I can fund it)
2. Confirmation that the bot is running
3. How to monitor it

Please proceed step-by-step and confirm each step before moving to the next.
```

What the AI Agent Needs From You

Before handing this to an agent, have these ready:

1. **Your Polymarket wallet address** (for `BROWSER_ADDRESS`)
2. **Your chosen copy target** (from leaderboard)
3. **Risk preferences** (conservative, moderate, aggressive)
4. **Funding ready**: USDC on Polygon to send to bot wallet

Example Agent Session

User:

```
Read guide/polymarket-copy-trading-masterclass.md and set up a bot copying prediction_wizard with
0.1x multiplier.
```

Agent:

```
I'll read the guide and set up your Polymarket copy trading bot step by step.

[Reads guide...]

Step 1: Generating a secure private key for your bot...
> openssl rand -hex 32
a1b2c3d4e5f6789012345678901234567890abcdef1234567890abcdef12345678

✅ Generated. Please save this key securely.

Step 2: Cloning repository...
> git clone https://github.com/...
✅ Cloned to /path/to/polymarket-copytrader

Step 3: Installing dependencies...
> uv sync
✅ Installed 23 packages

Step 4: Configuring .env...
PK=0xa1b2c3d4...
BROWSER_ADDRESS=[Your wallet address]
COPY_TRADE_TARGET=prediction_wizard
COPY_TRADE_SIZE_MULTIPLIER=0.1
MIN_SIZE=5
MAX_POSITION_USD=20

Step 5: Initializing database...
> uv run python scripts/init_sqlite_db.py
✅ Database initialized

Step 6: Starting bot...
> uv run python scripts/run_copytrade.py
✅ Bot is running!

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
📋 SETUP COMPLETE
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

Your bot wallet address: 0x9876543210987654321098765432109876543210

ACTION REQUIRED: Send at least $100 USDC (Polygon) to the address above.

Once funded, your bot will automatically copy prediction_wizard's trades.

Monitor with: tail -f bot_data/bot.log
```

See? The agent does all the work. You just fund the wallet and collect profits.

Appendix A: Glossary

Binary Outcome: A market with two possible outcomes (YES/NO).

CLOB: Central Limit Order Book. An exchange where buyers and sellers place limit orders at specific prices. Polymarket uses a CLOB.

Condition ID: A unique identifier for a prediction market on Polymarket.

Derived Credentials: Authentication credentials derived mathematically from a private key. Polymarket uses L2-derived credentials for CLOB trading.

Limit Order: An order to buy/sell at a specific price or better.

Market Order: An order to buy/sell immediately at the best available price.

Neg Risk: Negative risk markets where YES + NO prices don't sum to 1. Rare, but can happen.

PnL: Profit and Loss. Total gains or losses from trading.

Polygon: A scaling solution for Ethereum that enables fast, cheap transactions.

Position: An active trade in a market (e.g., "I have a YES position in the Bitcoin market").

Proxy Wallet: A smart contract wallet created by Polymarket when you connect your main wallet. Used for trading.

USDC: USD Coin, a stablecoin pegged to $1 USD. Used as collateral on Polymarket.

Win Rate: Percentage of trades that are profitable.

W/T: Wins divided by Total trades.

0x Address: An Ethereum/Polygon wallet address (starts with 0x).

Appendix B: Troubleshooting

Common Issues

Issue: Bot not copying any trades

Diagnosis:

1. Check if target is actively trading
2. Verify `COPY_TRADE_TARGET` is correct
3. Check bot logs for errors

Solutions:

```
# Check target activity
uv run python scripts/check_target_activity.py prediction_wizard

# Verify .env
cat .env | grep COPY_TRADE_TARGET

# Restart bot
```

```
pkill -f run_copytrade
uv run python scripts/run_copytrade.py
```

Issue: Bot is copying, but losing money

Diagnosis:

1. Check if target is also losing money
2. Verify `COPY_TRADE_SIZE_MULTIPLIER` is correct
3. Calculate your actual PnL

Solutions:

```
# Compare PnL
uv run python scripts/compare_pnl.py prediction_wizard

# Consider switching targets
# Visit polymarket.com/leaderboard and find a better trader
```

Issue: Bot keeps crashing

Diagnosis:

1. Check error logs
2. Verify all dependencies are installed
3. Check Python version

Solutions:

```
# Check Python version
python --version  # Should be 3.12+

# Reinstall dependencies
uv sync --reinstall

# Check logs
tail -100 bot_data/bot.log
```

Issue: Rate limited (429 errors)

Diagnosis:

1. Polling too frequently
2. Too many targets

Solutions:

```
# Edit .env
COPY_TRADE_POLL_SECONDS=60  # Increase from 30

# Or reduce targets
COPY_TRADE_TARGET=prediction_wizard  # Just one target
```

Issue: "Invalid signature" errors

Diagnosis:

1. PK is incorrect
2. Derived credentials are expired

Solutions:

```
# Verify PK
cat .env | grep PK

# Regenerate credentials
uv run python scripts/init_auth.py
```

Getting Help

Resources:

1. **Polymarket Discord**: discord.gg/polymarket
2. **GitHub Issues**: Post bugs in the repository
3. **Guide Updates**: Check for new versions

Debug Mode:

Enable verbose logging:

```
# Edit .env
LOG_LEVEL=DEBUG

# Restart bot
uv run python scripts/run_copytrade.py
```

Appendix C: Code Reference

Key Files in the Bot

`polymarket_client.py`

Handles all interaction with Polymarket's API and CLOB.

```
class PolymarketClient:
    def __init__(self, private_key: str):
        """Initialize client with private key"""
        self.pk = private_key
        self.address = derive_address(private_key)
        self.l2_auth = derive_l2_credentials(private_key)

    def get_leaderboard(self, limit: int = 100):
        """Fetch current leaderboard rankings"""
        response = requests.get(
            "https://api.polymarket.com/leaderboard",
```

```
            params={"limit": limit}
        )
        return response.json()

    def get_positions(self, address: str):
        """Get open positions for an address"""
        response = requests.get(
            f"https://api.polymarket.com/positions/{address}"
        )
        return response.json()

    def place_order(self, market_id: str, side: str, price: float, size: float):
        """Place a limit order on the CLOB"""
        order = create_order(market_id, side, price, size)
        signature = sign_order(order, self.l2_auth)

        response = requests.post(
            "https://clob.polymarket.com/orders",
            json=order,
            headers={"Authorization": f"Bearer {signature}"}
        )
        return response.json()
```

`copytrade.py`

Core copy trading logic.

```
class CopyTrader:
    def __init__(self, client, target_address: str, config: dict):
        self.client = client
        self.target_address = target_address
        self.config = config
        self.db = sqlite3.connect("bot_data/trades.db")

    def check_for_new_positions(self):
        """Check if target has opened new positions"""
        target_positions = self.client.get_positions(self.target_address)
        my_positions = self.client.get_positions(self.client.address)

        new_positions = [
            p for p in target_positions
            if p not in my_positions
        ]

        return new_positions

    def mirror_position(self, position):
        """Copy a position from the target"""
        # Calculate size based on multiplier
        original_size = position['size']
        my_size = original_size * self.config['size_multiplier']
```

```
        # Apply limits
        my_size = max(
            self.config['min_size'],
            min(my_size, self.config['max_position_usd'])
        )

        # Place order
        result = self.client.place_order(
            market_id=position['market_id'],
            side=position['side'],
            price=position['price'],
            size=my_size
        )

        # Save to database
        self.save_trade(position, result)

        return result

    def run(self):
        """Main copy trading loop"""
        while True:
            try:
                new_positions = self.check_for_new_positions()

                for position in new_positions:
                    if self.should_copy(position):
                        self.mirror_position(position)

                time.sleep(self.config['poll_seconds'])

            except Exception as e:
                log_error(e)
                time.sleep(60)
```

`run_copytrade.py`

Main bot entry point.

```
def main():
    # Load config
    config = load_env()

    # Initialize client
    client = PolymarketClient(config['PK'])

    # Get target address
    target_address = resolve_username(config['COPY_TRADE_TARGET'])

    # Start copy trader
    trader = CopyTrader(client, target_address, config)
```

```
    logger.info(f"🚀 Starting bot. Copying: {config['COPY_TRADE_TARGET']}")

    # Run forever
    trader.run()

if __name__ == "__main__":
    main()
```

How Authentication Works

Polymarket uses a custom authentication scheme:

```
def derive_l2_credentials(private_key: str) -> dict:
    """
    Derive L2 credentials for CLOB authentication
    """
    # 1. Get account from private key
    account = Account.from_key(private_key)

    # 2. Derive L2 key (simplified)
    l2_key = keccak(text=private_key + "polymarket-l2")

    # 3. Create signature
    signature = account.sign_message(l2_key)

    return {
        "address": account.address,
        "public_key": signature['publicKey'],
        "private_key": signature['signature']  # For signing orders
    }

def sign_order(order: dict, l2_creds: dict) -> str:
    """
    Sign an order with derived credentials
    """
    # 1. Hash order data
    order_hash = keccak(json.dumps(order, sort_keys=True))

    # 2. Sign hash
    signature = w3.eth.account.sign_message(
        encode_defunct(order_hash),
        private_key=l2_creds['private_key']
    )

    return signature['signature']
```

Conclusion

You now have everything you need to:

1. ✅ **Understand** Polymarket and copy trading strategy

2. ✅ **Set up** a fully automated copy trading bot
3. ✅ **Configure** risk management and sizing
4. ✅ **Monitor** and operate the bot
5. ✅ **Extend** with AI-powered signals (optional)
6. ✅ **Automate** the entire setup with AI agents

Next Steps:

1. Choose your copy target from the leaderboard
2. Decide your starting capital and risk tolerance
3. Follow the setup steps (or hand this guide to an AI agent)
4. Fund your bot wallet
5. Let the bot run and collect profits

Remember: Start small, verify everything works, then scale up gradually.

Questions? Check the troubleshooting section or join the Polymarket Discord community.

Built by vibestak — Building autonomous businesses

Last updated: January 2025 | Version: 1.0